



*INSTITUTO POLITÉCNICO
NACIONAL
ESCOM*

Bases de datos

Práctica 01, parte A: Investigación

Funes Santamaría José Roberto

Grupo: 3IV1

1.-¿Qué es un sistema de control de versiones y que problemas nos ayuda a solucionar?.

Un control de versiones es el proceso de seguimiento a los cambios o actualizaciones que se realizan sobre una base de datos, tiene como propósito evitar errores de código y pérdida de información, por lo que un sistema de control de versiones, por ejemplo: SQL Source Control, es una herramienta que registra los cambios hechos en el código base, ayuda a administrar los archivos de código SQL e incluso deshacer cambios no deseados.

El sistema de control de versiones nos ayuda a administrar los cambios y actualizaciones sin interrumpir el funcionamiento de la base de datos ni poner en riesgo los datos que guardamos en ella; facilita la administración de una base de datos sobre la cuál trabajan muchas personas ya que también permite ver quién y cuando realizó cambios en el código SQL.

2.-¿Cuál es la diferencia entre Git y GitHub?.

Git cumple todas las funciones de un sistema de control de versiones; GitHub es una plataforma de compilación de software, está basado en Git y trabaja mediante repositorios de Git subidos en la nube, por lo que permite a un equipo grande trabajar sobre un repositorio y además, ofrece herramientas que facilitan la coordinación de los programadores, por lo que funciona como una extensión de Git.

La principal diferencia es que Git es un sistema de control de versiones y GitHub es una herramienta de trabajo para Git, ya que, aunque cumple ciertas funciones similares, se usa más como un espacio de almacenamiento con herramientas de administración, pero el programa en el que está construido un proyecto sigue siendo Git.

3.-Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo.gitignore y archivo README.

-Repositorio: Es un espacio de guardado donde podemos almacenar ya sea localmente o en la nube según nuestras necesidades, cómo puede ser acceder a la carpeta de trabajo sin conexión a internet o bien compartir nuestro trabajo y permitir a otros usuarios conectarse, también nos permite observar cambios hechos a lo largo del tiempo; por ejemplo: documentos de Word compartidos por Teams para que todos los integrantes del equipo puedan trabajar sobre un mismo documento guardado en la nube.

-Confirmación (commit): Es un comando que hace un registro en el historial de Git de nuestro proyecto, guardando una copia de todos los archivos al realizar el cambio y permitiéndonos regresar para ver los cambios hechos y comparar si es que lo necesitamos; por ejemplo: es similar a cuando hacemos clic en “guardar” en un documento de Word para registrar el cambio y poder volver a ese punto si llegamos a equivocarnos.

-Rama (branch): Una rama crea apuntadores a copias realizadas por las confirmaciones (commit), lo que nos permite modificar o realizar pruebas en esos documentos sin hacerlo directamente sobre el original y que no se vea afectado, es como crear un proyecto “independiente” que en realidad es una copia del que ya estaba hecho.

-Fusión (merge): Es una combinación de dos ramas independientes en una sola que es posible hacer cuando Git encuentra una confirmación base en común de la última confirmación de la rama (commit), si eso resulta bien entonces crea una confirmación de fusión e intentará fusionar los historiales independientes.

-Conflicto de fusión: Es un problema que encuentra Git cuando en las dos ramas que intenta fusionar se encuentra el mismo archivo modificado y entonces Git no decidirá alguna de las dos opciones disponibles, sino que pedirá que el usuario lo haga.

-Pull request: Es una solicitud enviada por un programador al resto de programadores que trabajan en la base de datos para que discutan, retroalimenten y aprueben el código o modificaciones hechas antes de que quede integrada al código principal.

-Archivo gitignore: Son archivos que Git ignorará porque no son importantes o útiles, pueden ser archivos generados durante la compilación, archivos compilados, incluso también puede tratarse de archivos de configuración de IDE, nada realmente importante para git.

-Archivo README: Es el archivo de “presentación” del proyecto, incluye información como que funciones cumple, cómo funciona, como trabajar con él y también comentarios sobre el código.

4.- ¿Qué es el flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse?

El flujo de trabajo basado en ramas es una estrategia de trabajo que consiste en crear ramas paralelas a la rama principal, que en realidad son sobres estas ramas secundarias sobre las que se trabaja y que al comprobar que no introducirán errores se pueden fusionar con la principal. Las revisiones del código en pares se realizan para la depuración de errores, retroalimentación y evitar implementar algo que cause errores en la rama principal.